

Introduction to operations research

1 Course logistics

1.a About me

Hi, I'm [Guillaume Dalle](#), a researcher in applied mathematics at [LVMT](#). I work on various topics related to operations research, machine learning and high-performance computing, with applications to transport and logistics.

The easiest way to contact me is my academic email guillaume.dalle@enpc.fr or the Microsoft Teams messaging app. I will answer quickly unless we're outside of office hours, in order to preserve your personal life and mine. You can also come visit me in room A409 of the Bienvenüe building at École des Ponts, just remember to schedule it with me beforehand.

1.b Resources

- Course website on [Educnet](#).
- The website for [this group](#).

1.c Rules

- Be respectful towards your classmates and teacher: listen when they speak, raise your hand, etc.
- Arrive on time, send me an email if you can't.
- Feel free to ask me if you have any problems with this course.
- Language should not be an issue: English or French is fine with me.

1.d LLM policy

LLMs are amazing tools but learning requires friction. If you don't understand the ideas/math/code, you won't be able to supervise a robot doing the ideas/math/code.

My golden rule: interactions between humans (e.g. emails) should be written by humans. In class: there is absolutely no reason to use LLMs, you have me! If you would rather ask a chatbot, then I'm doing something wrong and I need to fix it. At home: use them *if you want* to further your understanding or assist with the project, but make sure this does not threaten your skill-building.

1.e Grading

- 50%: in-class written exam (pen and paper)
- 50%: oral presentation of your work during the class project (no written report)

2 Basics of optimization

2.a Problems

An optimization problem $\mathcal{P}$ is a family of inputs + a question to answer. For a decision problem, the answer is either **yes** or **no**. For an optimization problem, we seek the best candidate among a family.

An optimization problem is often formulated as

$$\min_x c(x) \quad \text{s.t.} \quad x \in \mathcal{X} \quad (1)$$

- “s.t.” means “subject to”
- x is the decision variable
- $\mathcal{X}$ is the set of feasible solutions
- $x \in \mathcal{X}$ is the set of constraints
- $c(x)$ is the criterion / objective / cost function

Solving the problem means finding the optimal value $\text{val}(P)$, and if possible, an optimal solution $x^* \in \text{argmin}\{c(x) : x \in \mathcal{X}\}$.

An instance $\mathcal{I}$ of a problem is one particular input with its numerical values.

2.b Algorithms and heuristics

An algorithm $\mathcal{A}$ is a sequence of elementary operations that can be implemented on a computer.

Time complexity $f(n)$ of an algorithm $\mathcal{A}$ = number of elementary operations necessary with input of size n . Polynomial algorithms are usually considered efficient, while exponential algorithms scare many researchers (not us).

There are several types of optimization algorithms:

- Exact algorithms: yield an optimal solution
- Approximation algorithms: yield a solution with bounded sub-optimality
- Heuristics: yield a solution with no guarantee at all

Local descent enables us to explore large solution spaces with small movements. Typical algorithm: given the current solution x_k

1. Compute and explore its neighborhood $\mathcal{N}(x_k)$
2. Pick a next solution $x_{k+1} \in \mathcal{N}(x_k)$ such that $c(x_{k+1}) < c(x_k)$

How to evaluate the quality of a solution without a certificate of optimality? Compare its value $c(x)$ with a lower bound $\ell \leq \text{val}(P) \leq c(x)$

2.c Goals of the class

TBD

3 Basics of graph theory

3.a Vocabulary

A graph $G = (V, E)$ is composed of:

- a set V of vertices, often called u or v , with $|V| = n$
- a set E of edges, often denoted by $e = (u, v)$, with $|E| = m$

A subgraph H is a couple $(V' \subset V, E' \subset E)$.

The degree of a vertex is the number of incidence edges.

The adjacency matrix $A = (A_{u,v}) \in \{0, 1\}^{n \times n}$ tells us whether two vertices are linked by an edge.

Exercise 3.3: powers of adjacency matrices.

3.b Paths

A path is a sequence of nodes linked by edges. Types of paths:

- simple: no edge is crossed twice
- elementary: no vertex visited twice
- cycle: start vertex = end vertex
- eulerian: crosses all edges once
- hamiltonian: visits all vertices once

3.c Types of graphs

- undirected: edges are not ordered $(u, v) = \{u, v\}$
- directed: edges are ordered $(u, v) = u \rightarrow v$ (we use the terminology “nodes” and “arcs”)
- simple: no duplicate edges or self loops
- complete: simple with all possible edges
- bipartite: split in two parts with all edges in the middle
- connected: all couples of vertices have a path between them
- eulerian / hamiltonian: contains a eulerian/hamiltonian cycle
- forest: contains no cycle
- tree: connected and contains no cycle

A graph is eulerian iff all its vertices have even degree (i.e. an even number of incident edges).

Exercise 3.5: criterion for Eulerian graphs.

3.d Zoo of graph problems

3.d.a Colorings and cliques

A coloring is a function $c : V \rightarrow \mathbb{N}$ such that two adjacent vertices do not share the same color: if $(u, v) \in E$, $c(u) \neq c(v)$. The minimum number of colors needed to color a graph is its chromatic number $\chi(G)$.

A clique is a complete subgraph. The maximum cardinality of a clique is denoted by $\omega(G)$.

We have $\omega(G) \leq \chi(G)$.

3.d.b Matchings and covers

A matching is a set of disjoint edges (maximum cardinality $\nu(G)$).

A vertex cover is a set of vertices that covers (has a member belonging to) every edge (minimum cardinality $\tau(G)$).

We have $\nu(G) \leq \tau(G)$.

3.d.c Stable sets and edge covers

A stable set is a set of vertices such that no two of them are linked to each other (maximum cardinality $\alpha(G)$).

An edge cover is a set of edges that covers (has a member incident to) every vertex (minimum cardinality $\rho(G)$).

We have $\alpha(G) \leq \rho(G)$.

Exercise 3.10: stable sets and vertex covers.

4 MILP modeling

A Linear Program is an optimization problem with a linear objective and linear constraints:

$$\min_{x \in \mathbb{R}^n} c^\top x \quad \text{s.t.} \quad Ax \leq b \quad (2)$$

A Mixed Integer Linear Program is an LP where some of the variables are constrained to be integers:

$$\min_{x \in \mathbb{Z}^p \times \mathbb{R}^{n-p}} c^\top x \quad \text{s.t.} \quad Ax \leq b \quad (3)$$

In theory, solving a MILP is hard. However:

- they are very useful to model lots of real-life problems
- there are practically efficient solvers that can handle millions of variables if the problem has a certain structure